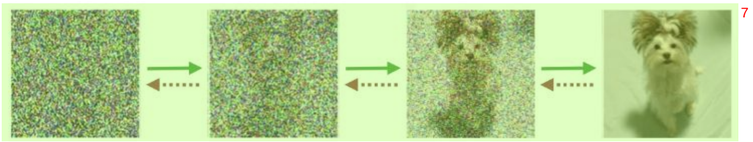
Learning to Reach Goals via Diffusion [1]

Vineet Jain Siamak Ravanbakhsh [2]

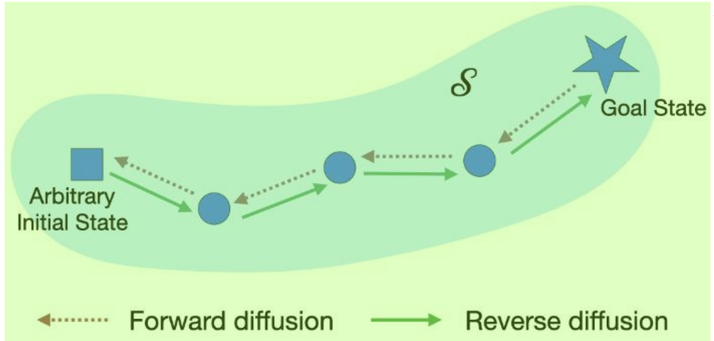
Generate trajectories that "diffuse away" from potential goals, and train a policy to reverse them [3]

Denoising Diffusion Models [4]



Noise Sample [8] Target Sample [9]

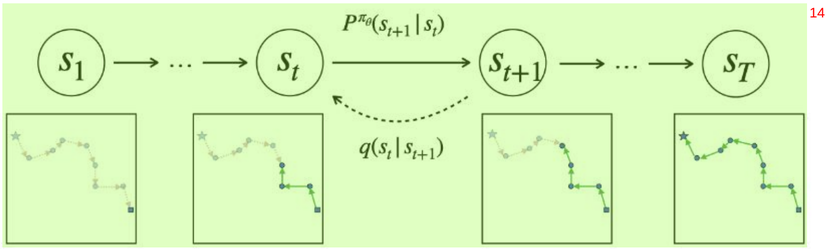
Goal-conditioned Reinforcement Learning [10]



[11]

Goal Conditioned RL as Diffusion [12]

Diffusion models: map any point from Gaussian → data manifold
Goal-conditioned RL: learn optimal path from any state → goal state [13]



[14]

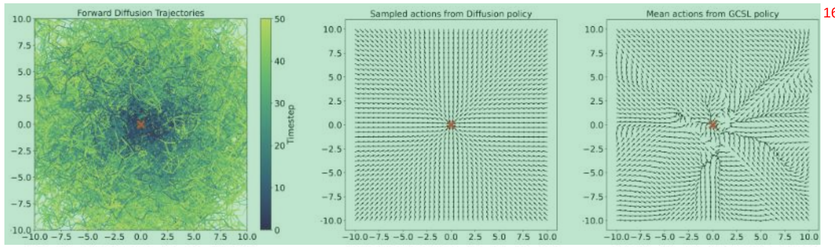
diffusion model	GCRL (Merlin)
target dataset	goal states
score function	policy
forward process	$q(s_t\|s_{t+1})$
reverse process	$\mathcal{P}^\pi(s_{t+1}\|s_t)$

[29]

Actions that lead us away from the goal in place of Gaussian noise; Policy that reverses these actions in place of score function [31]

A Simple 2D Navigation Problem [5]

- **Task:** Navigate to goal **X**; **States:** (x, y); **Actions:** unit length displacement in (x, y) [15]
- **Forward process:** Take random actions starting from goal
- **Reverse process:** Train policy to reverse the forward trajectories



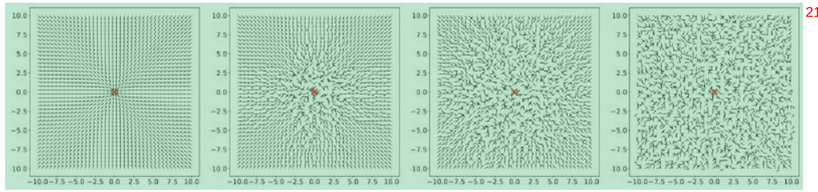
[16]

(a) Visualization of trajectories starting from the goal X generated during the forward process, (b) Predicted actions from Merlin policy, (c) Predicted actions from GCSL policy [17]

Backward view of diffusion gives us control over the goal distribution [18]

Effect of Time Conditioning [19]

- Analogous to the score function in diffusion, the Merlin policy is conditioned on the time [20]
- This time indicates how "noisy" current state is, or how far we are from the target goal state
- Policy takes optimal action far away from goal and becomes uncertain as it approaches the goal; this uncertainty appears farther from goal as time horizon increases



[21]

(a) h=1 (b) h=5 (c) h=10 (d) h=20 [22]

The time conditioning affects policy uncertainty based on expected number of steps to reach the goal [23]

Optimization Objective [28]

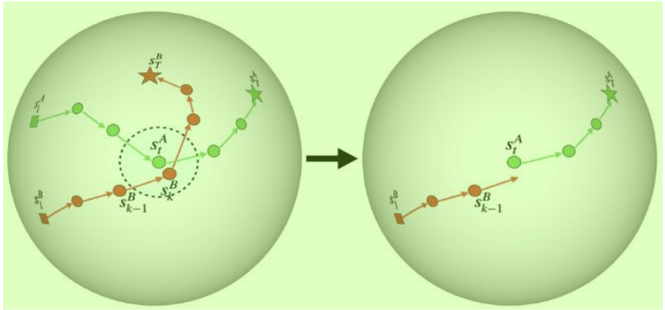
- Given an offline dataset $\mathcal{D}$, we can view the trajectories as forward diffusion chains generated by some unknown policy [32]
- As per below Theorem, behavior cloning maximizes the probability of reaching the goal states

$$\theta^* = \arg\max_\theta \mathbb{E}_{g\sim p(g),(s,a)\sim D(g)}[\log \pi_\theta(a \mid s, g)]$$

Theorem: Given a dataset $\mathcal{D}$ and target goal distribution $p(g)$, behavior cloning using a goal-conditioned policy maximizes a lower bound on the log-likelihood of the goal states $L = \mathbb{E}_{g\sim p(g), s_T\sim q(s_T|g)}[\log p_\theta(s_T)]$. [33]

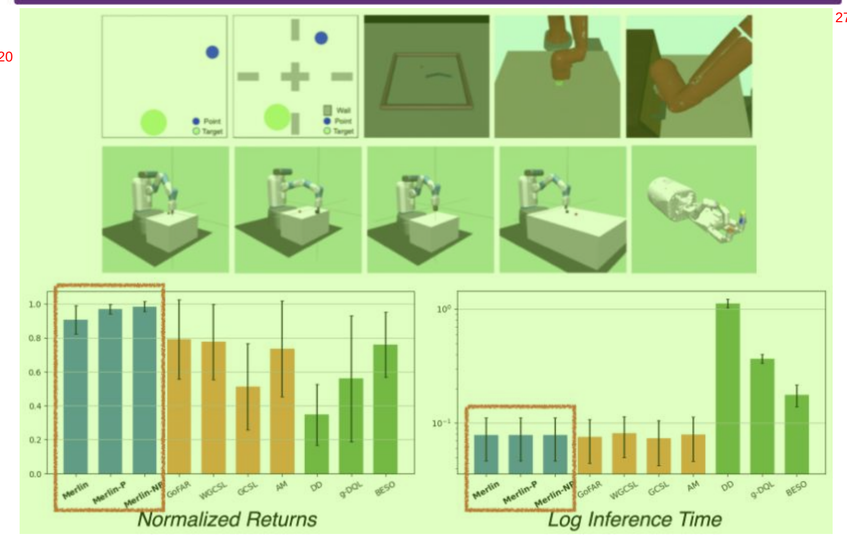
Constructing the Forward Process [6]

- **Merlin:** Use pre-collected offline trajectories, train policy using behavior cloning [24]
- **Merlin-P:** Learn reverse dynamics model to simulate trajectories backward from goal
- **Merlin-NP:** Use proposed trajectory stitching method to construct backward trajectories



[25]

Experimental Results [26]



[27]

Merlin outperforms previous methods, while being highly-efficient! [30]

Conclusions and future work [34]

- Merlin constructs trajectories that "diffuse away" from goals and trains a policy to reverse them [36]
- Simple training dynamics indicate potential for scalability, as proven by denoising diffusion models
- Directions for future work include extending to online setting and partially observable environments

paper code [35]